# TRINEXIS: An Alert-Aware Narration Policy and a Protocol for Measuring Its Effect on Alert Delivery

Yug G. Lakhani[1]
[1]Department of Computer Engineering, Gyanmanjari Innovative University, Bhavnagar, Gujarat, India
Corresponding author: Yug G. Lakhani, yglakhani@gmiu.edu.in
ORCID: https://orcid.org/0009-0004-2930-4953

**Abstract—** Open-vocabulary webcam systems must decide which detected objects to speak when output is short and time-limited. This report is framed around an alert-aware narration policy and its measured effect on alert delivery, which a future experiment must establish rather than assume. The policy ranks user-designated alert objects before truncating the spoken shortlist; a confidence-first baseline truncates by detector confidence first. The research question is whether alert-first narration makes users notice a designated object faster or more often than confidence-first narration. TRINEXIS implements the policy in an interval-throttled, language-configured webcam system and adds optional real-log instrumentation, a frozen alert manifest, Flask route tests, and continuous integration. Experiments A-D specify hardware timing, labeled detection evaluation, a vocabulary/confidence grid, real-log alert replay, and checked-language ratings; all empirical result cells remain NO DATA. The report therefore does not claim a measured user benefit, detection quality, latency, distance accuracy, multilingual comprehension, or assistive effectiveness. Its contribution is the policy, its failable software checks, and a preregistrable protocol for measuring alert delivery, not a description of a Flask application.

**Keywords / Index Terms—** alert-aware narration; open-vocabulary detection; research software; speech interface; empirical protocol; interval throttling; technical report

## I. INTRODUCTION

### A. Motivation and Research Question

Object detection predicts bounding boxes and semantic labels for objects in an image. The YOLO family formulated detection as a single regression problem [1], and COCO-style benchmarks standardized evaluation over a fixed vocabulary of 80 released categories [2]. Vision-language pre-training broadened the interface: CLIP enabled zero-shot transfer from natural-language supervision [3], Grounding DINO fused language and vision for open-set detection [4], and YOLO-World introduced a prompt-then-detect paradigm [5]. Ultralytics exposes that family through a set_classes interface [6]. These advances make configurable object names possible, but an interactive narration system still has to decide what to say first when only a few objects can be spoken.

The research question is: Does alert-first narration make users notice a designated object faster or more often than confidence-first narration? This report separates two measurable parts. Replayed real logs can measure whether an alert reaches the spoken shortlist and how long alert-to-speech delivery takes. Whether a person notices the object faster or more often requires a separate consented user study. Neither effect has been measured here; all empirical cells remain NO DATA.

TRINEXIS is the research instrument used to implement the policy: an interval-throttled YOLO-World webcam pipeline with a browser dashboard, language-configured speech output, session telemetry, and optional real-log instrumentation. Its maintained backend, app23.py, uses Flask control routes [7], OpenCV capture [8], Ultralytics YOLO [15], and gTTS speech [9]. The Flask application is implementation infrastructure, not the paper's contribution.

### B. Contribution

The contribution is an alert-aware narration policy and a protocol for measuring its effect on alert delivery, not a Flask app or a general webcam-application description. User-designated alert objects are ranked before the spoken shortlist is truncated. The ordering is isolated in a dependency-free policy module, checked by mutation-sensitive unit tests, instrumented for real-log replay, and deliberately kept separate from empirical detection or user-benefit claims.

- A policy and research question: alert-first versus confidence-first narration for a predesignated object, with delivery and noticing treated as distinct endpoints.
- A maintained implementation with AGPL-3.0 licensing [24], configurable camera/security settings, optional real-log timing and speech-event recorders, and a frozen alert-manifest mode.
- Real Flask test-client route tests, deterministic policy tests, continuous integration, and a source-level evidence-boundary check.
- Preregistrable Experiments A-D with hardware/data/rating manifests, runnable harnesses, and blank NO DATA result templates.

### C. Scope and Limitations

This manuscript is a standalone technical report with experiment protocols and blank empirical results; it is not submission-ready. No labeled detection dataset, hardware timing campaign, known-distance calibration, translated-template rating, consented user study, dashboard screenshot, or spoken input-to-output example was completed. No official venue template, current Guide for Authors, or citation style was supplied/read; generic single-column layout and numeric citations remain fallbacks. Funding and competing-interest statements require author confirmation, AI-tool disclosure must be rechecked against venue policy, and the author must personally verify every number. No detection-quality, latency, distance-error, robustness, multilingual-comprehension, assistive-effectiveness, or user-benefit result is claimed.

## II. RELATED WORK / LITERATURE REVIEW

### A. Detection and Open-Vocabulary Models

The original YOLO paper evaluated PASCAL VOC 2007/2012 and artwork-domain generalization on the Picasso and People-Art datasets; it did not evaluate on COCO [1]. COCO later supplied a fixed-vocabulary benchmark [2]. CLIP [3], Grounding DINO [4], YOLO-World [5], OWL-ViT [11], GLIP [12], Detic [13], and YOLOE [14] represent successive approaches to language-conditioned or open-vocabulary recognition. Their published results were obtained under distinct datasets, splits, prompts, hardware, and protocols, so they are not treated here as comparative baselines for TRINEXIS. Table I summarizes scope and non-comparability.

**TABLE I. VERIFIED RELATED WORK AND NON-COMPARABILITY**

| Source | Contribution | Verified evaluation context | Relevance and limitation |
|---|---|---|---|
| Redmon et al. [1], 2016 | One-stage YOLO detector | PASCAL VOC 2007/2012; Picasso and People-Art artwork generalization | Established real-time closed-set detection; no COCO evaluation in that paper and a different hardware/protocol context |
| Lin et al. [2], 2014 | Microsoft COCO benchmark | Approximately 328,000 images, 2.5 million labeled instances, 80 released categories | Standard fixed-vocabulary protocol; utils/coco.txt is only a label list in this repository |
| Radford et al. [3], 2021 | CLIP contrastive pre-training | 400 million image–text pairs; zero-shot transfer over more than 30 datasets | Representation learning, not a real-time bounding-box detector |
| Liu et al. [4], 2024 | Grounding DINO, ECCV 2024 published version | Open-set and referring-expression benchmarks under published protocols | Alternative open-set approach; not used in this repository |

| Source | Contribution | Verified evaluation context | Relevance and limitation |
|---|---|---|---|
| Cheng et al. [5], 2024 | YOLO-World and RepVL-PAN | CVPR 2024, pp. 16901–16911 | Direct model-family basis; published benchmark results are not reproduced here |
| Minderer et al. [11]; Li et al. [12]; Zhou et al. [13]; Wang et al. [14] | OWL-ViT, GLIP, Detic, YOLOE | ECCV/CVPR/arXiv protocols with distinct vocabularies and datasets | Expanded open-vocabulary context; incompatible with this repository's prompts and runtime |
| Alqahtani et al. [19], 2024 | Edge accuracy–latency–energy benchmark | YOLOv8, SSD, and EfficientDet Lite across heterogeneous edge devices | Motivates separate timing instrumentation; its measurements are not reused as TRINEXIS results |

## B. Assistive Interfaces, Speech, and Privacy

Remote sighted assistance has been studied through Be My Eyes [16], while Seeing AI [17] and AI scene-description applications [18] illustrate practical auditory presentation of visual information. These systems are relevant to the design space but do not validate TRINEXIS. The repository configures 72 language codes and 62 top-level speech resources. Ten language codes use an English template fallback in the UI; the remaining 72-language claim is unchecked. gTTS uses Google Translate's speech service [9], so raters should listen to sample audio and the author should listen to a few languages before any quality claim. Camera-based systems also raise privacy and face-recognition concerns [20], [21]. The optional face-recognition hook remains unavailable, and no consent, retention, or biometric-processing protocol is claimed.

## C. Research Software and Reproducibility

Git LFS stores small pointer files in Git while large model content is held elsewhere [10]. The prior immutable software release [23] includes CITATION.cff, codemeta.json, dependency-license notes, pinned manifests, and a weights download script that verifies Git LFS object identifiers as SHA-256 checksums. The unreleased working-tree revision adds experiment protocols, route tests, CI, and instrumentation that are not part of that release. The project adopts AGPL-3.0-only [24] because Ultralytics YOLO is distributed under AGPL-3.0 [22]. A Zenodo DOI has not been minted; the final GitHub/Zenodo release must come last because a DOI is permanent for the exact version.

# III. SYSTEM DESIGN

## A. Architecture and Control Flow

Figure 1 shows the runtime and control flow. A browser client calls Flask control routes and displays a Motion JPEG stream. A configurable camera backend supplies frames to OpenCV. Detector calls are gated by an interval throttle, after which YOLO-World produces text-conditioned detection records. Those records feed visual overlays, a relative distance index, a time-decayed heatmap, session telemetry, an alert-aware speech selector, and optional real-log recorders. Optional face recognition is shown as a dashed, unavailable path. There is no training or benchmark-evaluation path.

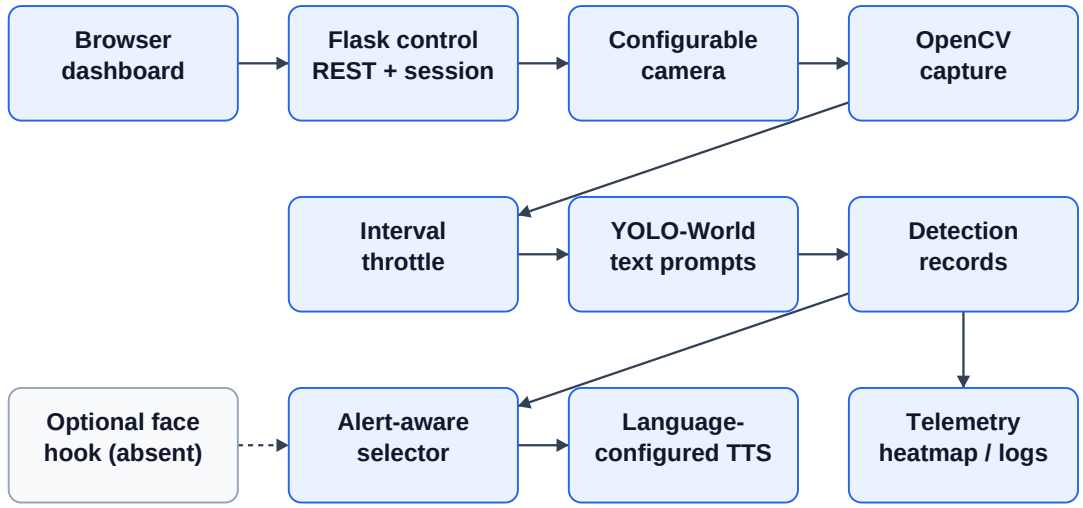


*Fig. 1. TRINEXIS system architecture. Solid arrows denote implemented runtime paths; the dashed arrow denotes an optional/unavailable face-recognition hook. Vector SVG/PDF versions are provided in the release artifacts.*

## B. Interval Throttle and Alert-Aware Narration

Let t be the current capture time, t_prev the previous detector invocation, and Delta the configured throttle interval. A detector call is admitted when

```
t - t_prev >= Delta. (1)
```

The default is Delta = 1.5 s and the spoken shortlist is limited to two objects. Let A be the set of user-designated alert names, c(o) the detector confidence, and d(o) the relative distance index. The selector sorts every detection record o by the ascending key

```
R(o) = (I_alert(o), -c(o), d(o)), (2)
```

and only then takes the first two records. In Eq. (2), I_alert(o) = 0 when o.name is in A and 1 otherwise; alert objects therefore receive priority regardless of whether they are among the two highest-confidence detections. A confidence-first shortlist followed by alert-aware reranking inside the shortlist would not provide this invariant. For Experiment C, alert names A are declared in a manifest before collection; the runtime freezes /set_alerts and embeds the manifest hash in detection and speech-event records.

## C. Time-Based Heatmap and Relative Distance Index

Eq. (3) replaces the older per-frame 0.995 multiplier, whose decay depends on camera frame rate, with elapsed wall-clock time dt and an adjustable half-life tau:

$$H_t = 2^{(-dt/\tau)} * H_{(t-1)} + \sum_i C(c_i; 35). \quad (3)$$

In Eq. (3), H_t is the heatmap at time t, C(c_i; 35) is a filled circle of radius 35 pixels centered on detection box c_i, and tau defaults to 60 s. The 60 s value is an adjustable default chosen to retain visible activity for roughly one minute; it is not fitted to data. The distance overlay is a unitless, relative size-based index

$$d_{idx} = (W * F) / \max(w_{px}, 1), \quad (4)$$

where w_px is the observed box width, W = 60, and F = 360. W and F are heuristic constants, not calibrated physical measurements; d_idx has no metric unit and must not be interpreted as centimetres or metres.

## D. Configuration, Security, Language Fallback, and Instrumentation

Camera index, backend, resolution, host, port, optional API token, read-only mode, single-client mode, client timeout, session secret, and heatmap half-life are read from environment variables. The default bind address is 127.0.0.1 rather than 0.0.0.0. /set_model accepts only a checked-in model allowlist. Mutating control endpoints can require an X-API-Token or Bearer header: missing or invalid tokens return 401, unsupported schemes and read-only mutations return 403, and a second controlling client receives 409. Capture-frame and detector-invocation rates are reported separately. Optional JSONL recorders remain off unless TRINEXIS_LATENCY_LOG and TRINEXIS_EVENT_LOG are configured; they timestamp capture, inference, TTS synthesis, playback, rendering, speech enqueue, and playback events. /get_languages exposes the English-fallback language list and an empty verified-language list.

**TABLE II. IMPLEMENTATION FEATURES AND VERIFICATION**

| Feature | Implementation | Verification |
|---|---|---|
| Open-source distribution | AGPL-3.0 LICENSE [24], README, pinned requirements, CITATION.cff, codemeta.json, and dependency-license notes | Source-contract tests and README instructions |
| Confidence display | Dashboard reports AVG CONFIDENCE rather than ACCURACY | Source-contract test |
| Alert-aware speech | Rank alerts before shortlist truncation via trinexis_core.select_speech_objects | Mutation-sensitive unit test and tests/ synthetic unit-test artifact |
| Client-state safety | Single-client control arbitration, 401/403/409 behavior, and stale-session timeout | Flask route tests and pure-function unit tests |
| Network defaults | Default host 127.0.0.1; optional API token and read-only mode on mutating control routes | Flask route tests and source-contract tests |
| Model loading | Allowlisted model filenames with HTTP 400 on rejection | Unit and route tests |
| Heatmap decay | Wall-clock exponential decay with adjustable half-life | Unit test |
| Distance display | Unitless relative distance index with explicit non-calibration note | Unit and source-contract tests |
| Camera portability | Configurable camera index and backend | Unit and source-contract tests |
| Rate reporting | Separate capture_fps and inference_fps counters and dashboard displays | Source-contract test |
| Language fallback | Ten missing templates fall back to English; 72 language codes remain unchecked | Route, language, and source-contract tests |
| Real-log instrumentation | Optional JSONL timing and speech-event recorders with manifest hashes | Telemetry unit tests and Experiment C protocol |
| Figures | Enlarged PNG plus standalone SVG and vector-PDF exports | make_updated_figures.py |

# IV. SOFTWARE VERIFICATION

## A. Verification Environment

All checks were static, synthetic, or exercised through Flask's test client with mocked native modules. The preparation environment used Python 3.11.2 with the standard library and Flask 3.1.3 for route tests; manuscript and figure generation used the pinned packages in paper/requirements-paper.txt. No model weights were hydrated, no camera was opened, no live Flask server was run, no network speech request was made, and no human participant was involved. Figure 2 summarizes the workflow. GitHub Actions is configured to run the same test suite on every push; a local CI-equivalent run passed, but no empirical result follows from a green test suite.

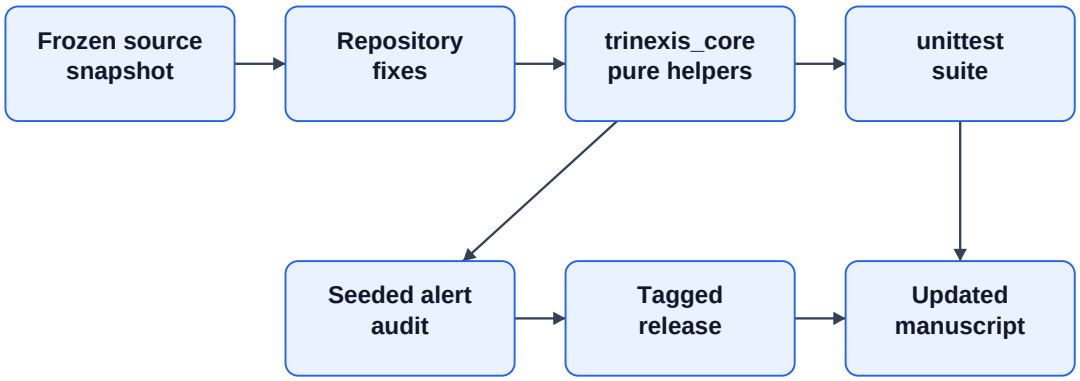


*Fig. 2. Software-verification workflow. Every step is static, synthetic, or a mocked Flask route test; no model inference, camera capture, labeled dataset, or human data is used. Vector SVG/PDF versions are provided in the release artifacts.*

## B. Synthetic Alert-Selection Unit Test

The seeded 1,000-case synthetic check is generated only by tests/test_synthetic_regression.py; the old experiments/ generator was removed. Each case has six records, one alert object, and a two-object speech shortlist. With six records and a top-2 shortlist, a uniformly ranked alert falls outside the shortlist with probability 4/6 (66.7%). The confidence-first ordering omitted the alert object in 681 of 1000 cases (68.1%), consistent with that probability setup; the alert-first ordering omitted it in 0 of 1000 cases (0.0%). A deliberately broken confidence-first selector makes the unit test fail. This is a failable unit-test artifact of an ordering invariant, not an experiment: it does not show how often real detections contain alerts, whether a real user hears an alert, or whether speech output is useful.

**TABLE III. SEEDED SYNTHETIC UNIT-TEST REGRESSION CHECK**

| Metric | Confidence-first | Alert-first | Change |
|---|---|---|---|
| Cases with a configured alert object | 1,000 | 1,000 | — |
| Alert omissions from the two-object shortlist | 681 | 0 | –681 |
| Omission rate | 68.1% | 0.0% | –68.1 pp |
| Evidence type | Synthetic unit test | Synthetic unit test | No model inference |

### C. Route, Policy, and Source-Contract Tests

The standard-library unittest suite contains 93 tests: 43 Flask route tests, 37 policy/harness/telemetry tests, and 13 source-contract tests. The exact CI statement is: 43 route tests plus 37 policy tests, run in CI. Route tests are reported separately from source-text tests. They exercise all 23 routes' status codes, JSON-object validation, clamping, API-token 401 behavior, unsupported-scheme/read-only 403 behavior, Bearer authentication, single-client 409 behavior, frozen alert manifests, and language-support metadata. Camera, NumPy, detector, and TTS imports are mocked with unittest.mock; no live runtime is validated by these counts. Policy tests cover narration ranking, mutation sensitivity, control status, telemetry arithmetic, language fallback, experiment-harness self-checks, and the seeded synthetic unit test. Continuous integration runs the suite on every push and pull request.

**TABLE IV. TEST-SUITE SUMMARY**

| Test group | Tests | Failures | Errors |
|---|---|---|---|
| Flask test-client route tests | 43 | 0 | 0 |
| Policy/harness/telemetry tests | 37 | 0 | 0 |
| Repository/source-contract tests | 13 | 0 | 0 |
| Total | 93 | 0 | 0 |

### D. Release and Reproducibility

The prior immutable software release is GitHub release v0.3.0-standalone [23]. The working-tree fix revision described here is unreleased; it adds CITATION.cff/codemeta metadata updates, experiment protocols, real-log instrumentation, route tests, CI, language-fallback documentation, and the reframed paper. weights/download_weights.py parses the checked-in Git LFS pointers and verifies the declared SHA-256 object identifiers after Git LFS materialization. Appendix A lists reproduction commands, and Appendix B lists the principal artifacts.

### E. Future Work

- Run Experiments A-D on declared hardware, labeled data, real logs, and checked languages; report negative results as-is.
- Obtain consent and institutional ethics approval before any user-noticing study.
- Calibrate the distance proxy at known distances or retain the unitless index with its current disclaimer.
- Add a dashboard screenshot, one spoken input-to-output example, and venue-policy/author-declaration checks before any submission.

## V. EMPIRICAL PROTOCOLS AND BLANK RESULTS

### A. Preregistration and Real Logs

All empirical cells are NO DATA. Before collection, the author must freeze hardware/software manifests, the paper and code, a 200-300-image report split, frozen prompt sets, and a predeclared alert-object manifest. The live app writes timestamped JSONL only when TRINEXIS_LATENCY_LOG and TRINEXIS_EVENT_LOG are configured. /export_logs supplies detection frames; event logs supply speech enqueue/playback records. The harnesses reject explicitly synthetic rows for empirical runs. Alert objects must be chosen in advance; the frozen manifest prevents post-hoc selection. No tuning on reported images is allowed, and the author must personally verify every number.

### B. Experiments A-D

Experiment A collects at least 500 camera frames per setting at 416, 640, and 832 input sizes on declared CPU/GPU/RAM/OS, Python, camera, audio, and TTS-backend configuration. It reports median and p95 for capture, inference, TTS synthesis, playback, and rendering. Experiment B evaluates 200-300 labeled COCO, LVIS, or own images with pycocotools or Ultralytics val, reporting precision, recall, mAP@0.5, and speed; its grid is {455, 20, 50, 80} prompts x {0.15, 0.25, 0.35}. Experiment C replays real detection logs through alert-first and confidence-first orderings with predeclared alert objects and reports delivery rate plus mean alert-to-speech delay. Experiment D samples 5-10 languages with native or reliable bilingual raters on 10 announcements each, rated 1-5 for correctness and naturalness; only checked languages may be reported, the remaining 72-language claims are unchecked, and raters should listen to gTTS/Google Translate speech samples. Negative or worse results are reportable.

### C. Blank Result Table

**TABLE V. EMPIRICAL PROTOCOLS AND BLANK RESULTS**

| Experiment | Primary endpoint | Prerequisite | Result |
|---|---|---|---|
| A. Latency | Median/p95 by stage and input size | Declared hardware; 500+ frames per 416/640/832 setting | NO DATA |
| B. Detection | Precision/recall/mAP@0.5/speed | 200-300 labeled images; separate tuning split; pycocotools or Ultralytics val | NO DATA |
| B. Grid | Twelve vocabulary/confidence cells | Frozen {455, 20, 50, 80} prompts x {0.15, 0.25, 0.35} | NO DATA |
| C. Alert replay | Delivery rate; mean alert-to-speech delay | Real /export_logs and speech events; predeclared alert manifest | NO DATA |
| D. Language | Mean correctness/naturalness | 5-10 checked languages x 10 announcements; 1-5 ratings; listen to samples | NO DATA |
| E. User noticing | Notice rate; time-to-notice | Consent and institutional ethics approval; optional 8-12 person within-subject study | NO DATA |

### D. What Replay Does Not Establish

Experiment C measures alert delivery from logs, not whether a person notices a designated object faster or more often. Answering the user-facing research question requires a separate consented, ethics-approved study with preregistered outcomes. No such approval, participant, or user benefit is claimed. The dashboard screenshot and spoken input-to-output example are also pending. A workshop or journal paper should be prepared only after these empirical steps are complete.

## VI. DISCUSSION

### A. Interpretation

The alert-aware policy makes the spoken shortlist independent of whether a designated object happens to be among the highest-confidence detections. Interval throttling separates detector cadence from camera frame rate; time-based heatmap decay and the unitless distance index remove two frame-rate- and calibration-dependent ambiguities. The Flask route tests establish control behavior under mocked dependencies, not live deployment success. Published detector literature is not comparable to this webcam pipeline because its datasets, prompts, splits, and hardware protocols differ. No measured effect on alert delivery or user noticing is reported.

### B. Ethical and Practical Implications

Camera-based interfaces can expose bystanders and sensitive environments. Privacy, consent, retention, and biometric-processing policies must be defined before any real-world or assistive-use study [20], [21]. The AGPL-3.0 license [24] and dependency-license notice clarify redistribution obligations but do not substitute for an ethics review. The system should not be described as assistive technology for blind or low-vision users until a consented study demonstrates benefit.

### C. Language Coverage and TTS

Seventy-two language codes and 62 top-level speech resources are configured. Ten language codes lack a top-level template and use the English fallback, which is now disclosed by /get_languages and the UI. No language is verified; no multilingual comprehension or translation-quality claim is made. gTTS uses Google Translate's speech service [9]. A checked-language study must use native or reliable bilingual raters, listen to sample audio, and report only the 5-10 languages actually rated; the author should also listen to a few languages. Missing templates may be added later only with reliable review; English fallback is documented rather than silently assumed.

## VII. CONCLUSION AND FUTURE WORK

This report defines an alert-aware narration policy, implements it in a configurable open-vocabulary webcam system, and supplies a protocol for measuring its effect on alert delivery. Software verification establishes the ordering invariant, HTTP control behavior, language fallback, and blank-result harnesses. The measured effect remains unknown: detection evaluation, hardware timing, real-log alert replay, checked-language ratings, and any user-noticing study are future work. The final release and Zenodo DOI must follow completed evidence and author verification, not precede them.

## DECLARATIONS

### A. Funding and Competing Interests

Author confirmation is pending; no approved funding or competing-interest statement is made in this working-tree draft. The author must verify these declarations before submission.

### B. Author Contributions (CRediT)

Draft CRediT assignment for author confirmation: Yug G. Lakhani—conceptualization, software, and writing—original draft. Methodology, formal analysis, investigation, validation, data curation, and review/editing must be matched to work actually performed after the empirical studies. No validation dataset or curated research data were used in this report.

### C. AI-Tool Disclosure

Arena.ai Agent Mode was used for source auditing, harness/test scripting, figure generation, repository scripting, and manuscript formatting. The author is responsible for the final manuscript, for rechecking this disclosure against the target venue's current AI-tool policy, and for verifying every number and declaration. No institutional ethics approval is claimed.

### D. Data and Code Availability

The prior immutable software release is available from GitHub release v0.3.0-standalone [23] under AGPL-3.0-only [24]; Ultralytics' AGPL-3.0 license is cited directly [22]. The unreleased working-tree revision on arena/69777d37-tech-yolo contains the reframed paper, experiment protocols, instrumentation, route tests, CI, and language-fallback UI; it is not part of that release. Model weights are Git LFS pointers and are fetched and checked with weights/download_weights.py [10]. No labeled dataset, experimental log, or valid Zenodo DOI is claimed. Zenodo DOI has not been minted because author authorization and the final GitHub/Zenodo release are pending; a DOI is permanent for the exact version. After that final release, insert the DOI into CITATION.cff, README.md, and this Data and Code Availability section, then rebuild the manuscript. Dependency licenses are documented in THIRD_PARTY_LICENSES.md. The generic layout and numeric citations remain fallbacks because no official venue template, current Guide for Authors, or citation style was supplied/read; this manuscript is not submission-ready.

# REFERENCES


[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[2] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740–755.

[3] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in Proc. Int. Conf. Mach. Learn. (ICML), PMLR 139, 2021, pp. 8748–8763.

[4] S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in Computer Vision – ECCV 2024, LNCS vol. 15105, 2024, pp. 38–55, doi: 10.1007/978-3-031-72970-6_3.

[5] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-World: Real-Time Open-Vocabulary Object Detection,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 16901–16911, doi: 10.1109/CVPR52733.2024.01599.

[6] Ultralytics, “YOLO-World documentation,” accessed Nov. 10, 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo-world/

[7] Flask Project, “Flask documentation,” accessed Nov. 10, 2026. [Online]. Available: https://flask.palletsprojects.com/

[8] OpenCV, “Getting Started with Videos,” accessed Nov. 10, 2026. [Online]. Available: https://docs.opencv.org/4.13.0/dd/d43/tutorial_py_video_display.html

[9] P. N. Durette, “gTTS: Python library and CLI tool to interface with Google Translate's text-to-speech API,” GitHub repository, accessed Nov. 10, 2026. [Online]. Available: https://github.com/pndurette/gTTS

[10] Git LFS Project, “Git LFS Specification: The Pointer,” accessed Nov. 10, 2026. [Online]. Available: https://github.com/git-lfs/git-lfs/blob/main/docs/spec.md

[11] M. Minderer et al., “Simple Open-Vocabulary Object Detection,” in Computer Vision – ECCV 2022, LNCS, 2022, pp. 728–755, doi: 10.1007/978-3-031-20080-9_42.

[12] L. H. Li et al., “Grounded Language-Image Pre-Training,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 10955–10965, doi: 10.1109/CVPR52688.2022.01069.

[13] X. Zhou, R. Girdhar, A. Joulin, P. Krähenbühl, and I. Misra, “Detecting Twenty-Thousand Classes Using Image-Level Supervision,” in Computer Vision – ECCV 2022, LNCS vol. 13669, 2022, pp. 350–368, doi: 10.1007/978-3-031-20077-9_21.

[14] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, “YOLOE: Real-Time Seeing Anything,” arXiv:2503.07465, 2025, doi: 10.48550/arXiv.2503.07465.

[15] Ultralytics, “Ultralytics YOLO,” GitHub repository, accessed Nov. 10, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[16] M. Avila, K. Wolf, A. Brock, and N. Henze, “Remote Assistance for Blind Users in Daily Life: A Survey about Be My Eyes,” in Proc. 9th ACM Int. Conf. Pervasive Technol. Related Assistive Environ. (PETRA), 2016, Article 85, 2 pages, doi: 10.1145/2910674.2935839.

[17] Microsoft, “Seeing AI,” accessed Nov. 10, 2026. [Online]. Available: https://www.seeingai.com/

[18] R. E. Gonzalez Penuela, J. Collins, C. L. Bennett, and S. Azenkot, “Investigating Use Cases of AI-Powered Scene Description Applications for Blind and Low Vision People,” in Proc. CHI Conf. Human Factors Comput. Syst. (CHI), 2024, Article 901, pp. 1–21, doi: 10.1145/3613904.3642211.

[19] D. K. Alqahtani, M. A. Cheema, M. A. Rodriguez, and A. N. Toosi, “A Comprehensive Evaluation of Deep Learning Object Detection Models on Heterogeneous Edge Devices,” arXiv:2409.16808, v1 2024, revised 2026, doi: 10.48550/arXiv.2409.16808.

[20] I. D. Raji, T. Gebru, M. Mitchell, J. Buolamwini, J. Lee, and R. Denton, “Saving Face: Investigating the Ethical Concerns of Facial Recognition Auditing,” in Proc. AAAI/ACM Conf. AI, Ethics, Soc. (AIES), 2020, pp. 145–151, doi: 10.1145/3375627.3375820.

[21] M. R. Hasan, R. Guest, and F. Deravi, “Presentation-level Privacy Protection Techniques for Automated Face Recognition—A Survey,” ACM Comput. Surv., vol. 55, no. 13s, Article 286, pp. 1–27, 2023, doi: 10.1145/3583135.

[22] Ultralytics, “LICENSE,” GitHub repository, AGPL-3.0, accessed Nov. 10, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics/blob/main/LICENSE

[23] Unusualyug, “tech_yolo release v0.3.0-standalone,” GitHub, 2026. [Online]. Available: https://github.com/Unusualyug/tech_yolo/releases/tag/v0.3.0-standalone

[24] Free Software Foundation, “GNU Affero General Public License, Version 3,” 2007. [Online]. Available: https://www.gnu.org/licenses/agpl-3.0.txt

## APPENDIX A. REPRODUCTION COMMANDS

The branch below is mutable and represents the unreleased working-tree revision, not the prior immutable tag.

```
git clone https://github.com/Unusualyug/tech_yolo.git
cd tech_yolo
git checkout arena/69777d37-tech-yolo
python3 -m pip install -r requirements-ci.txt
python3 -m unittest discover -s tests -v
python3 -m unittest tests.test_synthetic_regression -v
python3 scripts/report_test_counts.py
python3 experiments/latency_benchmark.py --self-check
python3 experiments/detection_eval.py --self-check
python3 experiments/vocab_confidence_grid.py --self-check
python3 experiments/alert_replay.py --self-check
python3 experiments/language_eval.py --self-check
python3 scripts/verify_no_fabricated_results.py
# Live studies only; CI does not install these or open hardware:
python3 -m pip install -r requirements.txt
python3 weights/download_weights.py --fetch --verify
python3 -m pip install -r paper/requirements-paper.txt
python3 paper/make_updated_figures.py
python3 paper/build_updated_manuscript.py
python3 paper/audit_updated_repository.py
```

## APPENDIX B. PRINCIPAL ARTIFACTS

- paper/tech_yolo_updated_research_paper.pdf, .docx, and .md: standalone technical report with blank empirical results.
- paper/artifacts/alert_selection_audit.json: tests/ unit-test artifact for the seeded synthetic alert-ordering regression.
- paper/artifacts/test_summary.json: route/policy/source-contract counts and exact CI report phrase.
- paper/artifacts/fig1_system_architecture_updated.{png,svg,pdf} and fig2_implementation_verification.{png,svg,pdf}: figure exports.
- paper/artifacts/updated_repository_audit.json: machine-readable working-tree/test summary.
- experiments/README.md and experiments/RESULTS_TEMPLATE.md: experiment protocols and blank NO DATA cells.
- experiments/latency_benchmark.py, detection_eval.py, vocab_confidence_grid.py, alert_replay.py, language_eval.py, and common.py: runnable harnesses.
- tests/test_routes.py, tests/route_test_support.py, and tests/test_synthetic_regression.py: real Flask route tests and the synthetic unit test.
- .github/workflows/ci.yml, requirements-ci.txt, scripts/report_test_counts.py, and scripts/verify_no_fabricated_results.py: CI and evidence-boundary checks.
- weights/download_weights.py and weights/weights_checksums.sha256: Git LFS pointer parser and SHA-256 manifest.
- zenodo.json and ZENODO.md: prepared Zenodo metadata and final-release DOI-minting instructions.
- paper/reference_verification.md: per-reference DOI/publisher verification log.
- FIX_STATUS.md, trinexis_core.py, trinexis_telemetry.py, trinexis_language.py, CITATION.cff, codemeta.json, README.md, LICENSE, requirements.txt, and THIRD_PARTY_LICENSES.md: software and status artifacts.